

Iniziato martedì, 16 dicembre 2025, 09:15**Stato** Completato**Terminato** martedì, 16 dicembre 2025, 10:20**Tempo impiegato** 1 ora 4 min.**Valutazione** 14,90 su un massimo di 18,00 (82,78%)Domanda **1**

Risposta errata

Punteggio ottenuto 0,00 su 1,00

Qual è un vantaggio del formato di istruzione fisso a 32 bit di RISC-V rispetto alle istruzioni a lunghezza variabile (come x86)?

Scegli un'alternativa:

- ☒ a. Fornisce una migliore compatibilità tra diverse implementazioni del processore ✖
- ☐ b. Permette di codificare valori immediati più grandi in tutte le istruzioni
- ☐ c. Elimina la necessità di allineamento delle istruzioni
- ☐ d. Riduce la dimensione totale del codice dei programmi
- ☐ e. Semplifica la logica di decodifica e fetch delle istruzioni

La risposta corretta è: Semplifica la logica di decodifica e fetch delle istruzioni

Domanda 2

Risposta errata

Punteggio ottenuto 0,00 su 1,00

Considerando un processore RISC-V a 32 bit, si supponga che il registro PC abbia il valore **0x00500000** (in esadecimale), e che l'istruzione corrente sia la prima istruzione riportata nel frammento di codice RISC-V di seguito.

Determinare l'indirizzo che sarà presente nel PC dopo l'esecuzione dell'istruzione **beq t0, zero, loop1**

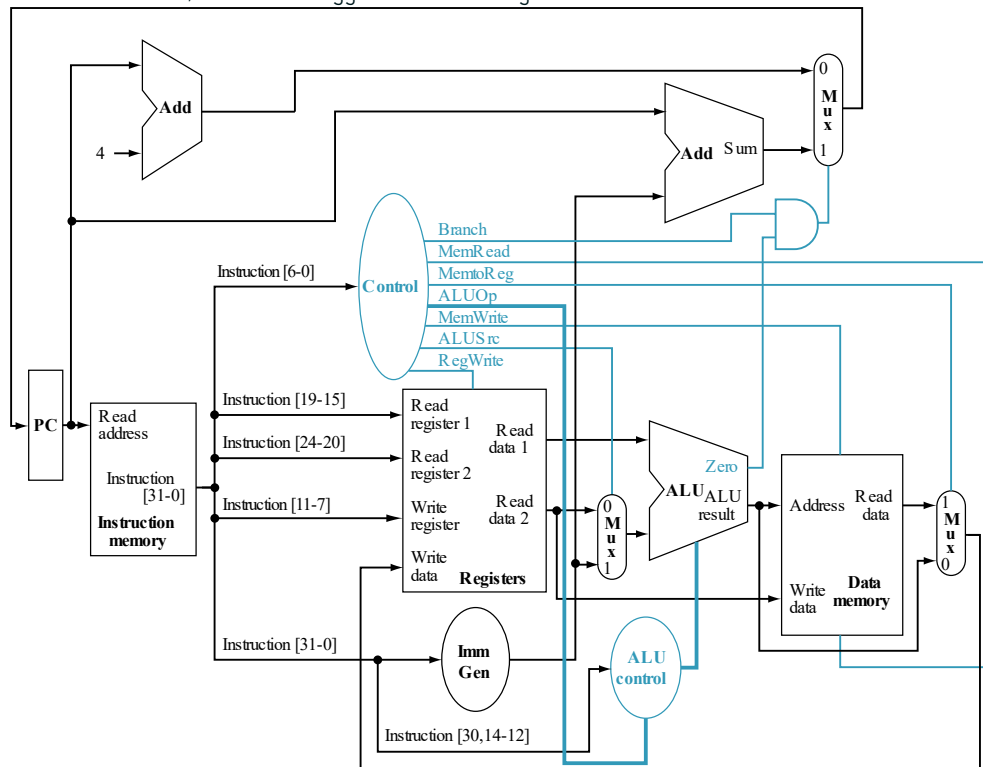
```

    addi t0, zero, 8
    addi t1, zero, 4
    or t0, t0, t1
    beq t0, zero, loop1
    addi t2, zero, 5
    jal zero, end

loop1:
    addi t2, zero, 10

end:
    nop
  
```

Come riferimento, il PC viene aggiornato dalla seguente rete nel RISC-V:



Scegli un'alternativa:

- ☒ a. **0x00500014** ✖
- ☐ b. **0x0050001C**
- ☐ c. **0x00500020**
- ☐ d. **0x00500010**
- ☐ e. **0x00500018**

La risposta corretta è: **0x00500010**

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considera la tabella con i formati di istruzione RISC-V mostrata nella figura seguente. Determina il tipo di istruzione e l'istruzione equivalente in linguaggio assembly per la seguente istruzione:

0100 0000 0011 0010 0101 0010 1011 0011

- (0,50 punti) Tipo di istruzione: ✓
- (0,50 punti) Istruzione assembly: ✓

RISC-V Instruction Set

Core Instruction Formats

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]				rs1		funct3		rd		opcode		I-type
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12:10:5]		rs2		rs1		funct3		imm[4:1:11]		opcode		SB-type
			imm[31:12]					rd		opcode		U-type
			imm[20:10:1:11:19:12]					rd		opcode		UJ-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith	R	0110011	0x5	0x20	rd = rs1 >> rs2	
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	msb-ext
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	signed unsigned
addi	ADD (Immediate)	I	0010011	0x0		rd = rs1 + imm	
xori	XOR (Immediate)	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR (Immediate)	I	0010011	0x6		rd = rs1 imm	
andi	AND (Immediate)	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[11:5]=0x00	rd = rs1 << imm[0:4]	
srl	Shift Right Logical Imm	I	0010011	0x5	imm[11:5]=0x00	rd = rs1 >> imm[0:4]	zero-ext
srai	Shift Right Arith Imm	I	0010011	0x5	imm[11:5]=0x20	rd = rs1 >> imm[0:4]	msb-ext
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	signed
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	unsigned
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-ext
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-ext
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	SB	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	SB	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	SB	1100011	0x4		if(rs1 < rs2) PC += imm	signed
bge	Branch ≥	SB	1100011	0x5		if(rs1 ≥ rs2) PC += imm	signed
bltu	Branch < (U)	SB	1100011	0x6		if(rs1 < rs2) PC += imm	unsigned
bgeu	Branch ≥ (U)	SB	1100011	0x7		if(rs1 ≥ rs2) PC += imm	unsigned
jal	Jump And Link	UJ	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Context switch to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Context switch to debugger	
fence	Memory fence	FENCE	0001111	0x0		Order mem/io accesses	

Domanda **4**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considerate un programma che elabora un array di grandi dimensioni sequenzialmente in un ciclo. Quale tipo di località sfrutta principalmente?

Scegli un'alternativa:

- ☐ a. Poca località temporale e nessuna località spaziale
- ☐ b. La località non è rilevante per l'elaborazione di array
- ☒ c. Principalmente località spaziale ✓
- ☐ d. Solo località temporale
- ☐ e. Località temporale e spaziale in modo uguale

La risposta corretta è: Principalmente località spaziale

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Due compilatori diversi hanno tradotto il seguente frammento di codice C:

```
if (a > 0) {  
    x = 1;  
}
```

Assumi che **a** sia in **t0** e **x** sia in **t1**.

Compilatore A:

```
1      bgtz t0, xset  
2      j skip  
3 xset: li t1, 1  
4 skip: nop
```

Compilatore B (incompleto):

```
1      _____  
2      li t1, 1  
3 skip: nop
```

Quale istruzione manca nel codice del Compilatore B (linea 1) per renderlo logicamente equivalente al Compilatore A?

Scegli un'alternativa:

- ☐ a. bgtz t0, skip
- ☒ b. blez t0, skip ✓
- ☐ c. mv t2, t0
- ☐ d. bnez t0, skip
- ☐ e. beqz t0, skip
- ☐ f. bltz t0, skip

La risposta corretta è: blez t0, skip

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si considera che la memoria principale contenga i valori riportati nella figura, dove il numero a sinistra della cella rappresenta l'indirizzo della cella.

Memoria

7	0x77
6	0x76
5	0x75
4	0x74
3	0x73
2	0x72
1	0x71
0	0x70

- Se il contenuto memorizzato dal registro **x4** fosse **0xFFFFFFFF** e se quello del registro **x5** fosse **0x00000004** quale sarebbe il valore (4 byte in esadecimale) del registro **x4** se il processore eseguisse l'istruzione **lhu x4, 2(x5)**?

0x00007776



- Se il contenuto memorizzato dal registro **x4** fosse **0x90919293** e se quello del registro **x5** fosse **0x00000004** quale sarebbe il valore del byte all'indirizzo 3 se il processore eseguisse l'istruzione **sh x4, -4(x5)**?

0x73



Riportare i risultati in esadecimale, senza alcun carattere di spaziatura (ad esempio: 0xFFFFFFFF o 0xFF).

Domanda 7

Parzialmente corretta

Punteggio ottenuto 0,40 su 1,00

Si supponga di avere una cache a **mappatura diretta** per un sistema con memoria indirizzabile al byte e indirizzi a 32 bit. Si supponga di voler implementare una cache con 8 KiB di dati, e che ogni blocco contenga 32 byte.

- (0.2 punti) Quante linee ha la cache? 256
- (0.2 punti) Quanti bit sono necessari per indirizzare le linee? 8
- (0.2 punti) Qual è la dimensione del tag in bit? 21
- (0.4 punti) Se si accede all'indirizzo 0x00002102, in quale linea della cache verrà caricato il blocco contenente questo indirizzo? 32

Domanda 8

Parzialmente corretta

Punteggio ottenuto 0,50 su 1,00

Si considerino tre diversi processori P1, P2 e P3 che eseguono lo stesso insieme di istruzioni.

- P1 ha una frequenza di clock di 1.0 GHz e un CPI (Cicli Per Istruzione) di 2.0.
- P2 ha una frequenza di clock di 2.0 GHz e un CPI di 2.5.
- P3 ha una frequenza di clock di 1.5 GHz e un CPI di 1.0.
- Quale processore ha le prestazioni migliori espresse in numero di istruzioni al secondo? ✖
- Si consideri un programma costituito da $1.0 \cdot 10^9$ istruzioni. Determinare il tempo (in secondi) richiesto da P1 per eseguire il programma. Nota: esprimere il risultato usando la virgola , come separatore tra la parte intera e quella frazionaria se necessario. Non inserire alcun carattere di spaziatura o unità di misura. ✔

Domanda 9

Risposta corretta

Punteggio ottenuto 5,00 su 5,00

Scrivi una procedura chiamata `has_digit(char *str)` che verifica se una stringa contiene almeno un carattere cifra ('0'-'9'). La funzione restituisce 1 se viene trovata almeno una cifra, o 0 altrimenti.

Ad esempio, se `str` contiene "hello123", la funzione restituisce 1. Se `str` contiene "hello", la funzione restituisce 0.

Il seguente codice in C implementa `has_digit` (convertilo in RISC-V):

```
int has_digit(char *str) {
    int i = 0;
    while (str[i] != '\0') {
        if (str[i] >= '0' && str[i] <= '9') {
            return 1;
        }
        i++;
    }
    return 0;
}
```

Si ricorda che il codice ASCII del carattere '0' è 48 e '9' è 57, e che la pseudo-istruzione `li t0, '0'` carica il valore 48 in `t0`.

Attenzione:

- Incollare solo la funzione `has_digit` (in RISC-V) nel campo sottostante
- Attenzione alle convenzioni di chiamata!
- Usare il seguente codice main per lo sviluppo e il debugging nel simulatore RARS

```
.globl _start
.data
    str: .string "hello123"

.text
_start:
    # chiama has_digit
    la  a0, str
    jal ra, has_digit

    li  a7, 10
    ecall

#####
# completa la funzione has_digit nel campo sottostante
```

Risposta (Criteri di penalizzazione: 0 %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?

Falling back to raw text area.


```
has_digit:
li t0, 0 #i=0
li t2, '9'
li t3, '0'

while:
lbu t1, 0(a0) #primo elemento
beq t1, zero, end_while
blt t1, t3, next_val #str[i] >= '0'
bgt t1, t2, next_val #str[i] <= '9'
li a0, 1 #return 1
ret

next_val:
addi a0, a0, 1 #prossimo indirizzo
addi t0, t0, 1 #i++
j while
```

	Test	Atteso	Ottenuto	
✓	<pre>.section .data str: .string "hello123"</pre>	a0: 1	a0: 1	✓

Tutti i test sono stati superati ✓

► Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 5,00/5,00.

Commento:

OK

Domanda **10**

Risposta corretta

Punteggio ottenuto 5,00 su 5,00

Scrivere una procedura chiamata `celsius_to_fahrenheit(int *temps, int n)` che riceve un array di temperature in Celsius `temps` e la sua dimensione `n`. La funzione deve convertire tutte le temperature da Celsius a Fahrenheit sostituendo ogni temperatura con il suo valore convertito.

La funzione `celsius_to_fahrenheit` deve utilizzare la funzione `to_fahrenheit` che riceve una temperatura in Celsius e restituisce la temperatura in Fahrenheit. Soluzioni che non utilizzano la funzione `to_fahrenheit` saranno considerate errate.

`to_fahrenheit` è già implementata in RISC-V e non deve essere riscritta. Una possibile implementazione di `to_fahrenheit` è fornita di seguito, ma il codice dovrebbe funzionare con qualsiasi altra implementazione purché segua le convenzioni di chiamata.

Il valore di ritorno deve essere lasciato nel registro `a0`.

Il seguente codice in C implementa `celsius_to_fahrenheit` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
void celsius_to_fahrenheit(int *temps, int n) {
    for (int i = 0; i < n; i++) {
        temps[i] = to_fahrenheit(temps[i]);
    }
}
```

Ad esempio, se `temps = [0, 20, 100]`, la funzione trasforma l'array in `[32, 68, 212]` perché:

0°C = 32°F; 20°C = 68°F; 100°C = 212°F

Attenzione:

- Incollare solo la funzione `celsius_to_fahrenheit` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice main per lo sviluppo e il debug nel simulatore RARS

```
.globl _start
.data
    temps: .word 0, 20, 100
    n:      .word 3

.text
_start:
    # chiama celsius_to_fahrenheit
    la    a0, temps
    la    a1, n
    lw    a1, 0(a1)
    jal   ra, celsius_to_fahrenheit

    li    a7, 10
    ecall

#####
# to_fahrenheit
# a0 -> temperatura in Celsius
# restituisce temperatura in Fahrenheit
#####
to_fahrenheit:
    li    t0, 9
    mul   t0, a0, t0      # celsius * 9
    li    t1, 5
    div   t0, t0, t1      # (celsius * 9) / 5
    addi  a0, t0, 32      # + 32
    ret

#####
# completa la funzione celsius_to_fahrenheit qui sotto
#####
```

Risposta (Criteri di penalizzazione: 0 %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```
celsius_to_fahrenheit:
addi sp, sp, -20
sw ra, 0(sp)
sw s1, 4(sp) #temps
sw s2, 8(sp) #n
sw s3, 12(sp) #i
sw s4, 16(sp) #temps[i]

li s3, 0 #i=0
mv s2, a1 #n
mv s1, a0 #temps
for:
bge s3, s2, end_for #i<n
lw s4, 0(s1) #temps[i]
mv a0, s4
jal to_fahrenheit
sw a0, 0(s1)
addi s3, s3, 1 #i++
addi s1, s1, 4 #indirizzo prossimo temperatura
j for

end_for:
lw ra, 0(sp)
lw s1, 4(sp)
lw s2, 8(sp)
```

	Test	Atteso	Ottenuto	
✓	.section .data temps: .word 0, 20, 100 n: .word 3	array = 32 68 212	array = 32 68 212	✓

Tutti i test sono stati superati ✓

▸ Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 5,00/5,00.

Commento:
OK